



ITERACIÓN

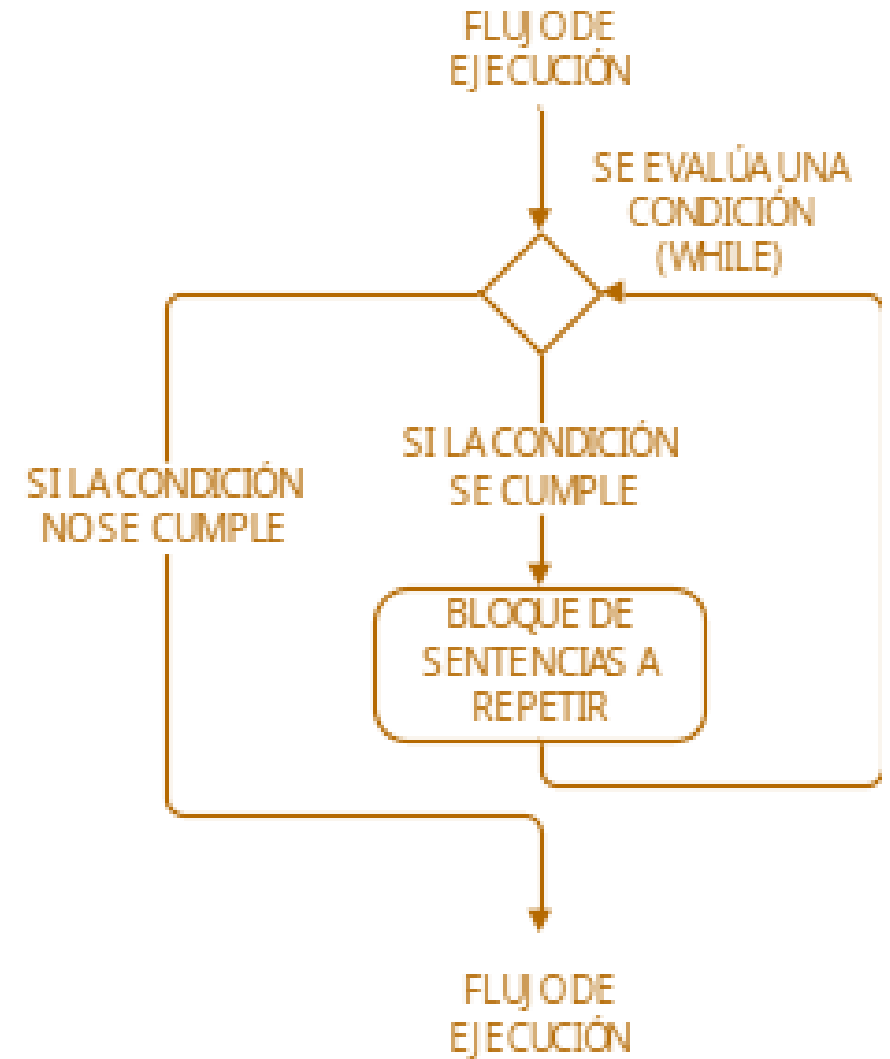
10145 - FUNDAMENTOS DE PROGRAMACIÓN PARA INGENIERÍA
10110 – FUNDAMENTOS DE COMPUTACIÓN Y PROGRAMACIÓN



RESUMEN DE CONTENIDOS

ITERACIÓN

- Flujo de una sentencia iterativa (**while**)
 - A diferencia de un **if**, cuando un bloque de sentencias condicionado por un **while** alcanza su fin **la condición se vuelve a evaluar**
 - Esto implica que el código puede **repetirse**
 - Pero en algún momento se debe **encontrar una forma de salir** del **while**





SENTENCIA **while**

- Estructura de la sentencia **while**

```
<Sentencias previas>  
while <condición>:  
    # Se ejecuta si la condición se cumple  
    <Bloque de sentencias a repetir>  
<Sentencias después del ciclo>
```



ACUMULADOR E ITERADOR

- **Acumulador:** la variable que guarda el resultado parcial
 - Se inicializa con el neutro de la operación: 0 para sumar, 1 para multiplicar
 - En cada vuelta se actualiza con su propio valor: $\text{suma} = \text{suma} + i$
- **Iterador:** la variable que avanza y hace que el ciclo termine
 - Con `while` hay que inicializarla y modificarla a mano
 - Con `for-in` la maneja Python: toma un valor distinto en cada vuelta
 - Si el iterador no cambia, la condición nunca deja de cumplirse: ciclo infinito

TRAZAS

- Analicemos el programa con **numero = 5**

```
numero = 5
```

```
i = 0
```

```
suma = 0
```

```
while i <= numero :
```

```
    suma = suma + i
```

```
    i = i + 1
```

```
# SALIDA
```

```
print ("La suma de los primeros", numero)
```

```
print ("números, es: ", suma)
```

Variable\Valor	Iteración						
	Inicio	1	2	3	4	5	6
numero	5	5	5	5	5	5	5
suma	0	0	1	3	6	10	15
i	0	1	2	3	4	5	6

TRAZAS

- Analicemos el programa con **numero = 5**

```
numero = 5
```

```
i = 0
```

```
suma = 0
```

```
while i <= numero :
```

```
    suma = suma + i
```

```
    i = i + 1
```

Variable\Valor	Iteración						
	Inicio	1	2	3	4	5	6
numero	5	5	5	5	5	5	5
suma	0	0	1	3	6	10	15
i	0	1	2	3	4	5	6

```
#  
p  
p
```

Al proceso de revisar manualmente el comportamiento del código con base en las variables involucradas, se le conoce como realizar una **traza del programa**

Y es una herramienta vital para entender qué es lo que hace un código



TAUTOLOGÍAS

- ¿Qué pasa si colocamos una tautología como condición?
 `i = 0`
 `while i < 10 :`
 `print('Debo practicar programación')`
- El mensaje 'Debo practicar programación' se repetirá por siempre
- El programa seguirá iterando hasta que **forcemos una detención**
 - En Colab la ejecución se interrumpe con el botón de detener de la celda, o en el menú Entorno de ejecución → Interrumpir ejecución
- Si tenemos una sentencia `while` de la cual es imposible salir durante una ejecución normal del programa significa que hemos creado **ciclo infinito**



MALAS PRÁCTICAS DEL CICLO

- En este curso no se usan `while True`, `break` ni `continue`
 - `while True` mueve la condición de salida adentro del ciclo: el encabezado deja de decir cuándo termina
 - `break` y `continue` rompen la traza: el flujo deja de poder seguirse línea a línea
- La forma del curso: la condición del `while` dice cuándo se sale

```
while i <= numero:
    suma = suma + i
    i = i + 1
```



SENTENCIA **for-in**

- La estructura **for-in** nos permite realizar iteración donde una variable irá tomando distintos valores para cada ciclo
- La **sintaxis** es la siguiente:

```
<sentencias previas>  
for <identificador> in <elemento iterable> :  
    <operaciones a realizar en el ciclo>  
  
<Sentencias siguientes>
```



COMPARANDO while CON for-in

```
suma = 0
i = 1
while i <= 5:
    suma = suma + i
    i = i + 1
print(suma)
```

```
suma = 0
for i in range(1, 6):
    suma = suma + i
print(suma)
```



FUNCIÓN `range()`

- La función `range()` tiene tres parámetros, uno mandatorio (obligatorio) y dos opcionales:
 - **Inicio:** Corresponde al primer valor del elemento iterable
 - Este parámetro es opcional y si se omite, se asume que el valor de inicio será 0
 - **Fin:** Corresponde al valor hasta el que esperamos llegar
 - Este parámetro **no puede omitirse** y `range()` creará elementos iterables hasta $\text{fin} - 1$
 - **Salto:** Indica, de cuánto es el salto entre cada elemento del iterable
 - Este parámetro es opcional y si se omite, se asume que el salto será 1.



EJERCICIOS



EJERCICIO PROPUESTO 1

- Construya un programa en Python que calcule la multiplicación de dos números enteros positivos usando solamente operadores de suma y resta. Por ejemplo:

$$- 5 * 4 = 5 + 5 + 5 + 5 = 4 + 4 + 4 + 4 + 4 = 20$$



EJERCICIO PROPUESTO 2

1. Modifique el código anterior para que calcule la división entera y el resto de números enteros positivos
1. Modifique el código anterior para que calcule la multiplicación utilizando cualquier par de números enteros (positivos o negativos)
1. Modifique el código anterior para que, en caso de que alguno de los números sea flotante, el programa lo informe y pida nuevamente el número



¿CONSULTAS?